

Trabajo Práctico N°1: Sistema de Gestión de Envíos y Distribución

Una empresa de logística está comenzando la digitalización de sus operaciones.

Actualmente, la empresa administra cientos de paquetes que deben ser preparados, clasificados, enviados y entregados a distintos destinos.

Hasta ahora todo se hacía en papel, lo que genera errores, pérdida de información y dificultades para seguir el estado real de cada envío.

Tu equipo fue contratado para desarrollar la primera versión del sistema orientado a objetos que permitirá modelar esta operación.

El objetivo no es simplemente almacenar datos, sino representar correctamente los objetos del dominio, sus responsabilidades y cómo colaboran entre sí.

Parte 1 — Modelado inicial

El sistema deberá representar, como mínimo, los siguientes objetos principales:

Paquete

Representa un objeto físico que será transportado.

Pistas:

- cada paquete posee identidad propia.
- puede contener información descriptiva.
- puede cambiar a lo largo del tiempo.
- no cualquier cambio de estado debería ser válido.

Preguntas para diseñar:

- ¿Qué información debería conocer un paquete?
- ¿Qué información debería permanecer protegida?

- ¿Quién debería poder modificar esa información?
- ¿Cómo cambia su estado?

Envío

Representa una operación logística que transporta de 1 a un máximo de 3 paquetes.

Pistas:

- un envío conoce aquello que transporta.
- puede tener un costo.
- puede variar según ciertas reglas del negocio.
- podría necesitar realizar cálculos internos.

Preguntas para diseñar:

- ¿Qué necesita conocer un envío para funcionar?
- ¿Quién debería calcular un costo?
- ¿Qué información debería ocultarse?

App/Main

Será utilizada para probar el sistema.

Debe permitir:

- crear objetos,
- relacionarlos,
- enviar mensajes entre ellos,
- mostrar resultados en consola.

Parte 2 — Encapsulamiento y estado

Todos los objetos deben proteger su información interna usando modificadores de acceso apropiados.

El sistema debe permitir que un paquete atraviese distintas etapas durante su ciclo de vida.

Ejemplos de situaciones:

- fue recibido,
- está en preparación,
- salió a distribución,
- fue entregado.

Preguntas para pensar:

- ¿El estado debería modificarse directamente?
- ¿Conviene usar setters para todo?
- ¿Qué acciones reales podrían provocar un cambio?

Parte 3 — Constructores y sobrecarga

Los objetos deben poder crearse en distintas situaciones.

Ejemplos:

Un paquete puede crearse:

- con información mínima,
- con información completa.

Un envío puede crearse:

- con datos básicos,
- con configuración más específica.

Desafío:

Diseñar constructores sobrecargados que tengan sentido para el negocio.

Pregunta guía:

- ¿Qué datos son obligatorios?
- ¿Qué datos pueden ser opcionales?

Parte 4 — Mensajes entre objetos

Los objetos deben colaborar entre sí.

Ejemplo de situaciones del negocio:

- un envío inicia su recorrido.
- al iniciar un envío, los paquetes involucrados deberían reflejar ese cambio.
- al finalizar un envío, los paquetes deberían actualizar su estado.

Desafío:

Resolver estas interacciones enviando mensajes entre objetos.

Pregunta clave:

Si algo debe ocurrir... ¿qué objeto debería tener esa responsabilidad?

Parte 5 — Lógica interna y ocultamiento

Algunas reglas del negocio no deberían quedar expuestas públicamente.

Ejemplos:

- validar pesos.
- calcular costos.
- aplicar recargos.
- verificar si una transición es válida.

Desafío:

Crear comportamiento interno utilizando métodos **private** cuando corresponda.

Pregunta guía:

¿Este comportamiento necesita ser usado por otros objetos o solamente por el propio objeto?

Parte 6 — Colecciones

La empresa ahora necesita registrar múltiples envíos durante una jornada.

Desafío:

Modificar el sistema para que la aplicación pueda trabajar con varios envíos simultáneamente usando una lista. Esta lógica, **por ahora**, la dejaremos en el main/app.

La aplicación deberá permitir:

- registrar múltiples envíos,
- recorrerlos,
- mostrar información resumida.

Parte 7 — Evolución con herencia

La empresa quiere incorporar distintos tipos de envío:

- estándar,
- express,
- internacional.

Cada tipo puede calcular costos de manera distinta.

Desafío:

Evitar decisiones del tipo: `if(tipo.equals(...))`. Necesitamos usar **soluciones orientada a objetos**.

Pregunta guía:

¿Cómo lograr que distintos objetos respondan al mismo mensaje de formas diferentes?

Preguntas de reflexión

Las preguntas se responden en un “readme” dentro del proyecto. Las mismas son de reflexión y pensadas para hacerlas personales sin uso de AI.

1. ¿Qué responsabilidades asignaste a cada objeto?
2. ¿Qué decisiones te resultaron difíciles?
3. ¿Qué métodos decidiste hacer privados y por qué?
4. ¿Qué setters evitaste implementar y por qué?
5. ¿Qué código tuviste que cambiar al incorporar herencia?